

进程是由内核自动调度的，而且因为它们有各自独立的虚拟地址空间，所以要实现共享数据，必须要有显式的 IPC 机制。事件驱动程序创建它们自己的并发逻辑流，这些逻辑流被模型化为状态机，用 I/O 多路复用来说明地调度这些流。因为程序运行在一个单一进程中，所以在流之间共享数据速度很快而且很容易。线程是这些方法的混合。同基于进程的流一样，线程也是由内核自动调度的。同基于 I/O 多路复用的流一样，线程是运行在一个单一进程的上下文中的，因此可以快速而方便地共享数据。

无论哪种并发机制，同步对共享数据的并发访问都是一个困难的问题。提出对信号量的 P 和 V 操作就是为了帮助解决这个问题。信号量操作可以用来提供对共享数据的互斥访问，也对诸如生产者-消费者程序中有限缓冲区和读者-写者系统中的共享对象这样的资源访问进行调度。一个并发预线程化的 echo 服务器提供了信号量使用场景的很好的例子。

并发也引入了其他一些困难的问题。被线程调用的函数必须具有一种称为线程安全的属性。我们定义了四类线程不安全的函数，以及一些将它们变为线程安全的建议。可重入函数是线程安全函数的一个真子集，它不访问任何共享数据。可重入函数通常比不可重入函数更为有效，因为它们不需要任何同步原语。竞争和死锁是并发程序中出现的另一些困难的问题。当程序员错误地假设逻辑流该如何调度时，就会发生竞争。当一个流等待一个永远不会发生的事件时，就会产生死锁。

参考文献说明

信号量操作是 Dijkstra 提出的 [31]。进度图的概念是 Coffman [23] 提出的，后来由 Carson 和 Reynolds [16] 形式化的。Courtois 等人 [25] 提出了读者-写者问题。操作系统教科书更详细地描述了经典的同步问题，例如哲学家进餐问题、打瞌睡的理发师问题和吸烟者问题 [102, 106, 113]。Butenhof 的书 [15] 对 Posix 线程接口有全面的描述。Birrell [7] 的论文对线程编程以及线程编程中容易遇到的问题做了很好的介绍。Reinders 的书 [90] 描述了 C/C++ 库，简化了线程化程序的设计和实现。有一些课本讲述了多核系统上并行编程的基础知识 [47, 71]。Pugh 描述了 Java 线程通过内存进行交互的方式的缺陷，并提出了替代的内存模型 [88]。Gustafson 提出了替代强扩展的弱扩展加速模型 [43]。

家庭作业

- 12.16 编写 hello.c (图 12-13) 的一个版本，它创建和回收 n 个可结合的对等线程，其中 n 是一个命令行参数。
- 12.17 A. 图 12-46 中的程序有一个 bug。要求线程睡眠一秒钟，然后输出一个字符串。然而，当在我们的系统上运行它时，却没有任何输出。为什么？

code/conc/hellobug.c

```

1  /* WARNING: This code is buggy! */
2  #include "csapp.h"
3  void *thread(void *vargp);
4
5  int main()
6  {
7      pthread_t tid;
8
9      Pthread_create(&tid, NULL, thread, NULL);
10     exit(0);
11 }
12
13 /* Thread routine */
14 void *thread(void *vargp)
15 {
16     Sleep(1);
17     printf("Hello, world!\n");
18     return NULL;
19 }
```

code/conc/hellobug.c

图 12-46 练习题 12.17 的有 bug 的程序

B. 你可以通过用两个不同的 Pthreads 函数调用中的一个替代第 10 行中的 `exit` 函数来改正这个错误。选哪一个呢？

- * 12.18 用图 12-21 中的进度图，将下面的轨迹线分类为安全或者不安全的。

A. $H_2, L_2, U_2, H_1, L_1, S_2, U_1, S_1, T_1, T_2$

B. $H_2, H_1, L_1, U_1, S_1, L_2, T_1, U_2, S_2, T_2$

C. $H_1, L_1, H_2, L_2, U_2, S_2, U_1, S_1, T_1, T_2$

- ** 12.19 图 12-26 中第一类读者-写者问题的解答给予读者的是有些弱的优先级，因为读者在离开它的临界区时，可能会重启一个正在等待的写者，而不是一个正在等待的读者。推导出一个解答，它给予读者更强的优先级，当写者离开它的临界区的时候，如果有读者正在等待的话，就总是重启一个正在等待的读者。

- ** 12.20 考虑读者-写者问题的一个更简单的变种，即最多只有 N 个读者。推导出一个解答，给予读者和写者同等的优先级，即等待中的读者和写者被赋予对资源访问的同等的机会。提示：你可以用一个计数信号量和一个互斥锁来解决这个问题。

- ** 12.21 推导出第二类读者-写者问题的一个解答，在此写者的优先级高于读者。

- ** 12.22 检查一下你对 `select` 函数的理解，请修改图 12-6 中的服务器，使得它在主服务器的每次迭代中最多只回送一个文本行。

- ** 12.23 图 12-8 中的事件驱动并发 `echo` 服务器是有缺陷的，因为一个恶意的客户端能够通过发送部分的文本行，使服务器拒绝为其他客户端服务。编写一个改进的服务器版本，使之能够非阻塞地处理这些部分文本行。

- * 12.24 RIO I/O 包中的函数(10.5 节)都是线程安全的。它们也都是可重入函数吗？

- * 12.25 在图 12-28 中的预线程化的并发 `echo` 服务器中，每个线程都调用 `echo_cnt` 函数(图 12-29)。`echo_cnt` 是线程安全的吗？它是可重入的吗？为什么是或为什么不是呢？

- ** 12.26 用加锁-复制技术来实现 `gethostbyname` 的一个线程安全而又不可重入的版本，称为 `gethostbyname_ts`。一个正确的解答是使用由互斥锁保护的 `hostent` 结构的深层副本。

- ** 12.27 一些网络编程的教科书建议用以下的方法来读和写套接字：和客户端交互之前，在同一个打开的已连接套接字描述符上，打开两个标准 I/O 流，一个用来读，一个用来写：

```
FILE *fpin, *fpout;
```

```
fpin = fdopen(sockfd, "r");
fpout = fdopen(sockfd, "w");
```

当服务器完成和客户端的交互之后，像下面这样关闭两个流：

```
fclose(fpin);
fclose(fpout);
```

然而，如果你试图在基于线程的并发服务器上尝试这种方式，将制造一个致命的竞争条件。请解释。

- * 12.28 在图 12-45 中，将两个 V 操作的顺序交换，对程序死锁是否有影响？通过画出四种可能情况的进度图来证明你的答案：

情况 1		情况 2		情况 3		情况 4	
线程 1	线程 2	线程 1	线程 2	线程 1	线程 2	线程 1	线程 2
P(s)	P(s)	P(s)	P(s)	P(s)	P(s)	P(s)	P(s)
P(t)	P(t)	P(t)	P(t)	P(t)	P(t)	P(t)	P(t)
V(s)	V(s)	V(s)	V(t)	V(t)	V(s)	V(t)	V(t)
V(t)	V(t)	V(t)	V(s)	V(s)	V(t)	V(s)	V(s)

- * 12.29 下面的程序会死锁吗？为什么会或者为什么不会？

初始时: $a = 1, b = 1, c = 1$

线程1:	线程2:
P(a);	P(c);
P(b);	P(b);
V(b);	V(b);
P(c);	V(c);
V(c);	
V(a);	

- 12.30 考虑下面这个会死锁的程序。

初始时: $a = 1, b = 1, c = 1$

线程1:	线程2:	线程3:
P(a);	P(c);	P(c);
P(b);	P(b);	V(c);
V(b);	V(b);	P(b);
P(c);	V(c);	P(a);
V(c);	P(a);	V(a);
V(a);	V(a);	V(b);

- 列出每个线程同时占用的一对互斥锁。
- 如果 $a < b < c$, 那么哪个线程违背了互斥锁加锁顺序规则?
- 对于这些线程, 指出一个新的保证不会发生死锁的加锁顺序。

- ** 12.31 实现标准 I/O 函数 `fgets` 的一个版本, 叫做 `tfgets`, 假如它在 5 秒之内没有从标准输入上接收到一个输入行, 那么就超时, 并返回一个 NULL 指针。你的函数应该实现在一个叫做 `tfgets-proc.c` 的包中, 使用进程、信号和非本地跳转。它不应该使用 Linux 的 `alarm` 函数。使用图 12-47 中的驱动程序测试你的结果。

```

1  #include "csapp.h"
2
3  char *tfgets(char *s, int size, FILE *stream);
4
5  int main()
6  {
7      char buf[MAXLINE];
8
9      if (tfgets(buf, MAXLINE, stdin) == NULL)
10         printf("BOOM!\n");
11     else
12         printf("%s", buf);
13
14     exit(0);
15 }

```

code/conc/tfgets-main.c

code/conc/tfgets-main.c

图 12-47 家庭作业题 12.31~12.33 的驱动程序

- ** 12.32 使用 `select` 函数来实现练习题 12.31 中 `tfgets` 函数的一个版本。你的函数应该在一个叫做 `tfgets-select.c` 的包中实现。用练习题 12.31 中的驱动程序测试你的结果。你可以假定标准输入被赋值为描述符 0。
- ** 12.33 实现练习题 12.31 中 `tfgets` 函数的一个线程化的版本。你的函数应该在一个叫做 `tfgets-thread.c` 的包中实现。用练习题 12.31 中的驱动程序测试你的结果。
- ** 12.34 编写一个 $N \times M$ 矩阵乘法核心函数的并行线程化版本。比较它的性能与顺序的版本的性能。
- ** 12.35 实现一个基于进程的 TINY Web 服务器的并发版本。你的解答应该为每一个新的连接请求创建一个新的子进程。使用一个实际的 Web 浏览器来测试你的解答。
- ** 12.36 实现一个基于 I/O 多路复用的 TINY Web 服务器的并发版本。使用一个实际的 Web 浏览器来测试你的解答。

- ** 12.37** 实现一个基于线程的 TINY Web 服务器的并发版本。你的解答应该为每一个新的连接请求创建一个新的线程。使用一个实际的 Web 浏览器来测试你的解答。
- ** 12.38** 实现一个 TINY Web 服务器的并发预线程化的版本。你的解答应该根据当前的负载，动态地增加或减少线程的数目。一个策略是当缓冲区变满时，将线程数量翻倍，而当缓冲区变为空时，将线程数目减半。使用一个实际的 Web 浏览器来测试你的解答。
- ** 12.39** Web 代理是一个在 Web 服务器和浏览器之间扮演中间角色的程序。浏览器不是直接连接服务器以获取网页，而是与代理连接，代理再将请求转发给服务器。当服务器响应代理时，代理将响应发送给浏览器。为了这个试验，请你编写一个简单的可以过滤和记录请求的 Web 代理：
- 试验的第一部分中，你要建立以接收请求的代理，分析 HTTP，转发请求给服务器，并且返回结果给浏览器。你的代理将所有请求的 URL 记录在磁盘上一个日志文件中，同时它还要阻塞所有对包含在磁盘上一个过滤文件中的 URL 的请求。
 - 试验的第二部分中，你要升级代理，它通过派生一个独立的线程来处理每一个请求，使得代理能够一次处理多个打开的连接。当你的代理在等待远程服务器响应一个请求使它能服务于一个浏览器时，它应该可以处理来自另一个浏览器未完成的请求。
- 使用一个实际的 Web 浏览器来检验你的解答。

练习题答案

- 12.1 当父进程派生子进程时，它得到一个已连接描述符的副本，并将相关文件表中的引用计数从 1 增加到 2。当父进程关闭它的描述符副本时，引用计数就从 2 减少到 1。因为内核不会关闭一个文件，直到文件表中它的引用计数值变为零，所以子进程这边的连接端将保持打开。
- 12.2 当一个进程因为某种原因终止时，内核将关闭所有打开的描述符。因此，当子进程退出时，它的已连接文件描述符的副本也将被自动关闭。
- 12.3 回想一下，如果一个从描述符中读一个字节的请求不会阻塞，那么这个描述符就准备好可以读了。假如 EOF 在一个描述符上为真，那么描述符也准备好可读了，因为读操作将立即返回一个零返回码，表示 EOF。因此，键入 Ctrl+D 会导致 select 函数返回，准备好的集合中有描述符 0。
- 12.4 因为变量 pool.read_set 既作为输入参数也作为输出参数，所以我们在每一次调用 select 之前都重新初始化它。在输入时，它包含读集合。在输出，它包含准备好的集合。
- 12.5 因为线程运行在同一个进程中，它们都共享相同的描述符表。无论有多少线程使用这个已连接描述符，这个已连接描述符的文件表的引用计数都等于 1。因此，当我们用完它时，一个 close 操作就足以释放与这个已连接描述符相关的内存资源了。
- 12.6 这里的主要的思想是，栈变量是私有的，而全局和静态变量是共享的。诸如 cnt 这样的静态变量有点小麻烦，因为共享是限制在它们的函数范围内的——在这个例子中，就是线程例程。
- A. 下面就是这张表：

变量实例	被主线程引用?	被对等线程0引用?	被对等线程1引用?
ptr	是	是	是
cnt	否	是	是
i.m	是	否	否
msgs.m	是	是	是
myid.p0	否	是	否
myid.p1	否	否	是

说明：

- ptr：一个被主线程写和被对等线程读的全局变量。
- cnt：一个静态变量，在内存中只有一个实例，被两个对等线程读和写。
- i.m：一个存储在主线程栈中的本地自动变量。虽然它的值被传递给对等线程，但是对等线程也绝不会在栈中引用它，因此它不是共享的。